ConfigStream

Evidence-oriented configuration history and streaming analytics

Phạm Công Hoàng Lê Tiến Hợp
Mai Vũ Hoàng Nguyễn Đức Dũng
Group 4 · Big Data Storage & Processing

Project evidence review · 26 October 2026



SOICT

The problem: arrival order can change the answer

Router configuration: version 10

```
hostname router-001
logging enabled
telnet disabled
```

Version 11

```
hostname router-001
logging disabled
telnet enabled
```

The real problem is pairing states, not storing files

Snapshots arrive as messages. Version 2 can arrive after version 3, so arrival order cannot define the transition.

ConfigStream must order and validate temporal state before it computes a diff.

Illustrative input from the executable demo: processing/demo.py.

If temporal state is wrong, every downstream result is wrong

Late arrival

v1, v3, v2 can produce v1 → v3 if arrival order is trusted.

Missing state

A missing state can look like a deletion or a real configuration change.

Uncommitted result

A projection can show a result before the authoritative history is committed.

Therefore: temporal state first, diff second, publication third.

Input snapshots → ordered state → normalized diff → committed history → query

Failure modes formalized in the state, lake, and domain acceptance documents.

A trustworthy history needs four guarantees

1. **Identity** links every observation to an asset and source coordinate.
2. **Producer order** determines adjacency; transport arrival order does not.
3. **Completeness** prevents the system from inferring a change from missing input.
4. **Recovery evidence** lets readers rebuild projections from committed history.

These guarantees define the solution before any technology is chosen.

Common evidence contract, stateful processing policy, and lake-first publication boundary.

Four datasets, four meanings of change

Terraform / TerraDS: versions

Infrastructure code stored in Git.

v1 → **v2**: which lines changed?

RPKI: observed daily states

Observed route-authorization records.

Day 1 → **day 2**: which members differ?

RIS: routing events

BGP announcements and withdrawals.

Announce → **withdraw**: what happened?

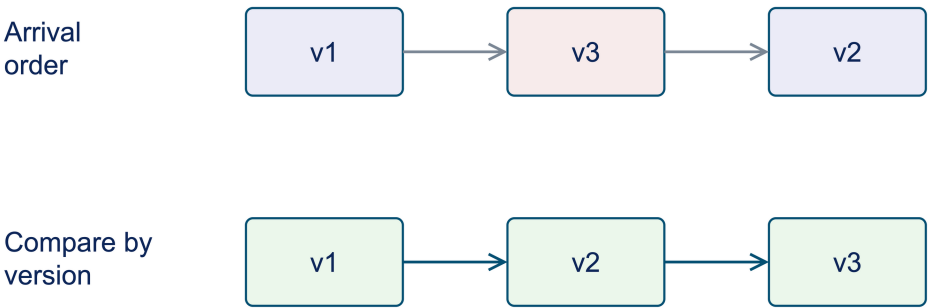
DNS: measurements

Recorded DNS query and response data.

Name → **IP**: what was observed, when?

Conceptual examples. Sources and accepted scopes: frontend/public/research/v1/catalog.json.

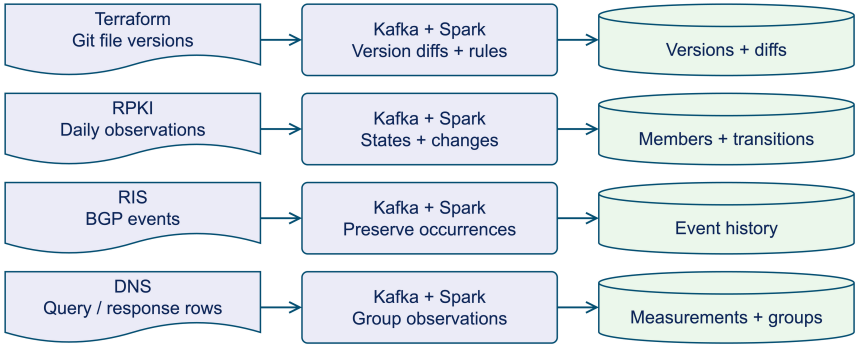
Configuration example: handling late arrivals



Compare **v1** → **v2** and **v2** → **v3**, once both endpoints are available.

processing/state.py; spark/streaming/stateful.py. Online repair is bounded by retained state.

Solution: separate temporal state from publication



Kafka + Spark + MinIO (Parquet), with separate schemas and processing.
Config search: Elasticsearch + FastAPI. Dashboard: accepted results from all four.

spark/streaming/; spark/rpki_stream.py; spark/ris.py; spark/dns.py; domain architecture docs.

Evidence scope: four domains, four units

Dataset	Accepted amount	Selected scope
Terraform	**10,188** Git versions	6,505 file histories / 375 repositories
RPKI	**14** complete daily observations	ripencc.tal, 1–14 Sep 2026
RIS	**4,725,156** event occurrences	rrc00, 1 Sep 2026, 00:00–01:00 UTC
DNS	**2,234,863** raw measurements	Two .li source objects, 1–2 Sep 2026

Controlled synthetic inputs also test retries and late arrivals.
Source manifests and result receipts are versioned in Git.

Accepted evidence: frontend/public/research/v1/catalog.json and its linked validation artifacts. Units differ; do not sum the rows.

Implementation: deterministic diffs and rule alerts

Diff: version 10 → 11

```
- logging enabled
- telnet disabled
+ logging disabled
+ telnet enabled
```

Rules detect

Logging disabled
High severity

Telnet enabled
Critical severity

Repeated delivery must not invent another logical change.

processing/demo.py; processing/diff.py; rules/default.yaml; tests/unit/test_rules.py.

Implementation evidence: code, tests, and a runnable demo

Repository

`generator/`	input data
`processing/`	diff and rules
`spark/`	streaming jobs
`api/`	read API
`frontend/`	dashboard

Small demo, no services required

```
make setup
make processing-demo
```

178 core tests passed
State logic, diffs and rules
Rechecked on 25 October 2026

github.com/kongwoang/ConfigStream; Makefile; tests/unit/test_state_logic.py, test_diff.py, test_rules.py.

Evidence: the integrated local pipeline completed



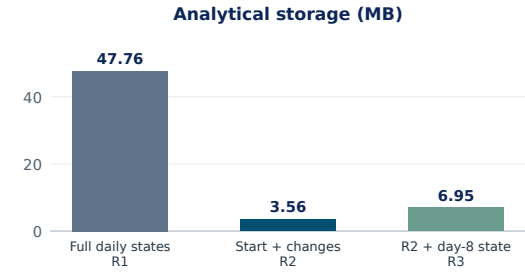
29.4 MB of raw and processed Parquet committed.

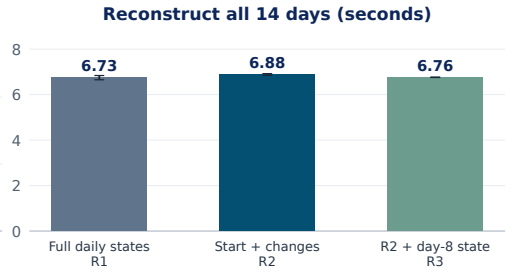
The integrated path processed the selected workload and committed the resulting history.

Recorded acceptance: docs/phase12-acceptance.md. 29,402,997 bytes committed.

RPKI: less storage, similar reconstruction time

Same 14 daily observations. All three layouts reconstruct the exact same states.





R2 saves 92.6% of analytical storage.

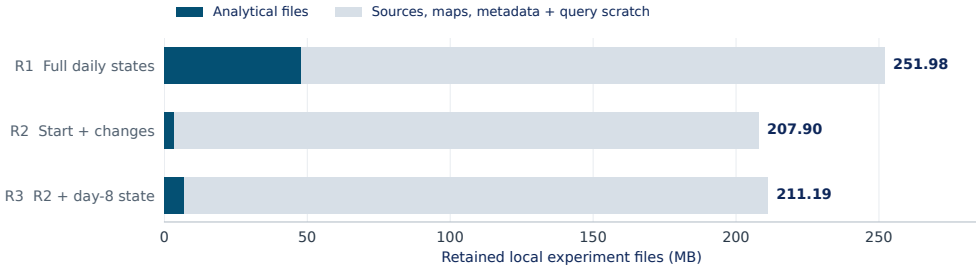
Reconstructing all 14 days still takes about 6.9 seconds.

Recorded local E2R experiment: workloads/results/phase20/rpki-representations.json. MB = 1,000,000 bytes.

Time: median of 3 build medians, each with 3 warm cycles; whiskers show the range of build medians. Independent verification timed separately.

The storage saving depends on what we count

Same RPKI experiment, now including the other retained files.



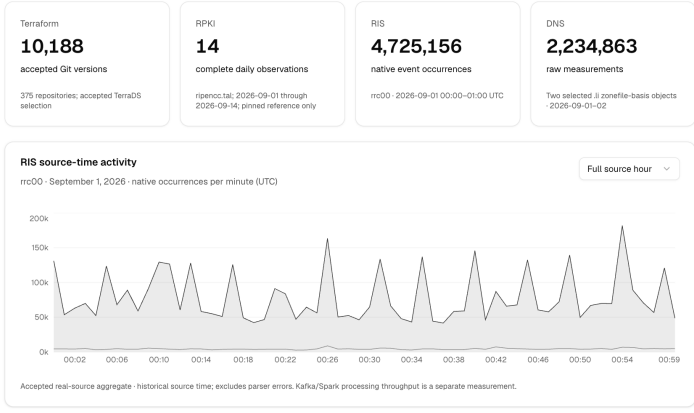
R2 saves 17.5% across measured local experiment files.

Sources, provenance maps and query scratch still occupy most of the space.

workloads/results/phase20/rpki-all-in-storage.json. Median accounted content bytes across 3 builds per layout.

Includes analytical files, sources, coordinate maps, metadata and retained scratch.

Evidence surface: history, experiments, and provenance



Phase 20V frontend capture from the accepted evidence bundle, 25 October 2026.

What the evidence supports

Requirement	What the group can show
Git + executable code	Repository, runnable demo and automated tests
Data ready	Terraform, RPKI, RIS, DNS and source manifests
Architecture ready	Four processing paths, shared storage and dashboard

The core workflow is complete within the declared scope

Conclusion: complete within the declared scope

- The system preserves identity, producer order, completeness, and provenance across four data domains.
- The accepted evidence includes an integrated replay, a matched 27-trial campaign, storage observations, and a rebuildable read path.
- Each result remains tied to its domain, source manifest, acceptance receipt, and declared processing boundary.

Thank you. Questions?

Backup: accepted result ledger

Evidence item	Value	Interpretation
TerraDS snapshots	10,188	Accepted local replay input
Adjacent diffs	3,683	Version-ordered transitions
Replay output	29,402,997 bytes	Raw and processed Parquet
Alerts / diagnostics	0 / 0	In the accepted replay
Matched trials	27	9 configurations, 3 fresh runs each
Retained growth	3,869,485,576 bytes	Below the amended 6 GiB limit
Aggregate processing	502.981 s	Below the 7,200 s acceptance limit

Acceptance receipts and phase 20 matched campaign. Values are recorded evidence, not extrapolated capacity.

Backup: matched processing campaign

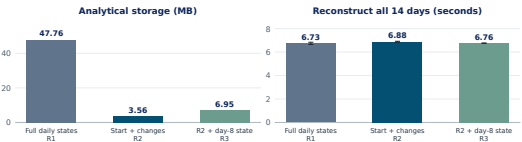
Workload	B0	B2	B3
RPKI, 360,226 rows	6.297	16.840	25.417
RIS, 370,141 occurrences	10.485	21.167	29.644
DNS, 4,096 rows	2.089	10.536	16.866

What the table supports

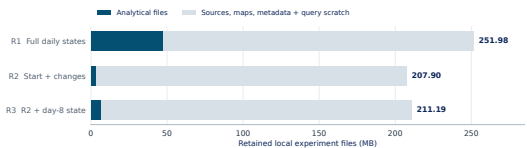
All accepted trials produced identical normalized rows. The comparison is a component-level timing result under a normalized-core workload.

Medians in seconds; observed ranges are reported in the written report.

Backup: RPKI storage trade-off



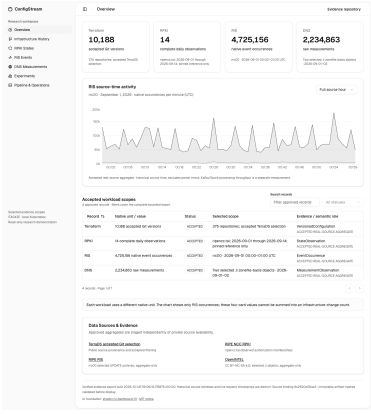
Analytical files: R2 saves 92.6%

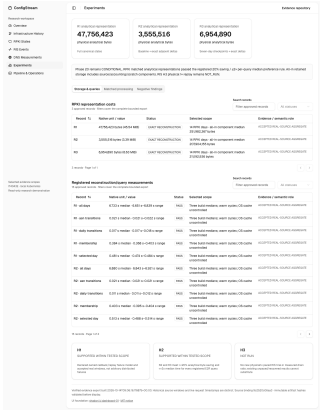


All measured files: R2 saves 17.5%

The denominators differ. The broader accounting includes sources, coordinate maps, metadata and query scratch.

Backup: interface evidence



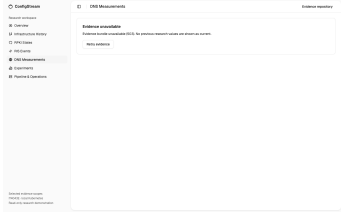


Phase 20V frontend captures. They document the read-only evidence surface; they do not prove a live backend replay at capture time.

Backup: responsive and failure states







Responsive layout and explicit error presentation are part of the interface review evidence.

Backup: limits that must remain visible

- The accepted evidence is local and single-node.
- The RIS result is an occurrence-processing result, not a complete global BGP state.
- The DNS result retains coverage and vantage uncertainty.
- The configuration method is line-oriented for the supported representation, not a universal parser.
- The lake-first boundary supports rebuildable projections, but it does not prove end-to-end exactly-once processing.

Threats-to-validity record and domain-specific acceptance documents.